

ECE servers how-to

ECE Linux computing facility

The ECE department provides several Linux servers:

- eceterm1, eceterm2, eceterm3 are used only for accessing the other machines from outside the campus firewall
- eceubuntu1, eceubuntu2, eceubuntu4
- ecetesla0, ecetesla1, ecetesla2, ecetesla3, ecetesla4 (several are equipped with NVIDIA GPUs)

Note 1: See <https://ece.uwaterloo.ca/Nexus/arbeau/clients/> for load statistics.

Note 2: Use TCP port numbers in the range 10,000 - 11,000.

Note 3: To speed up access to ecelinux hosts, please use SSH public key authentication, set up an SSH agent and enable agent forwarding.

ECE Linux computing facility

- ecehaoop is a 16-node (4 cores per node) cluster which is setup to run Hadoop/Spark and MPI jobs.
 - ecehadoop is the master node, ecehadoop0.private.uwaterloo.ca - ecehadoop15.private.uwaterloo.ca are the worker nodes. (Include the "StrictHostKeyChecking=no" ssh option to avoid permission issues that may prevent your jobs from running.)
- If you want to run Hadoop/Spark jobs on ecehaoop, please let the instructor know, an account will be created for you.
- Storage space on the cluster is limited, so please try keep your HDFS usage as low as possible.
- If you want to experiment with large inputs then consider placing them in HDFS under /tmp where they can be accessed by other group members/students. This avoids data duplication.
- Please limit yourself to running one Spark job at a time. Do not let any job run for hours.

ECE Linux computing facility

Checking GPU/CUDA availability on a server:

- For example, `ssh userID@ecetesla2.uwaterloo.ca`
- run the following command to check GPU availability: `"nvidia-smi"`
- (ecetesla2 is equipped with NVIDIA GeForce RTX 3070)
- run the command to check CUDA version: `"nvcc --version"`

ECE Linux computing facility

- ece-nebula cluster has hosts with high core count, large memory and modern GPUs.
- Mike Cooper-Stachowsky (mstachowsky@uwaterloo.ca) manages ece-nebula cluster, they can give you access to the cluster.

MPI

- The following set of slides describe how to run an MPI application on eceubuntu and ecetesla servers, and ecehadoop cluster.

Using the starter code

- The MPI starter code tarball is available [here](#).
- The starter code contains an MPI application written in C++.
- In this MPI application, rank 0 reads two square matrices of the same order and performs matrix multiplication locally.
- Also, rank 0 sends the order of the input matrices to another rank using a blocking send operation.
- When the computation is completed, rank 0 writes the result of matrix multiplication to a file.
- If the program is launched with more than one MPI processes, other ranks will just wait for rank 0 at the barrier.
- The tarball also includes input matrices and code for generating ground truth (i.e., output of matrix multiplication).

Using the starter code

- You may add your code in the “matrix_multiplication.cpp” file (as indicated in the starter code).
- You may add new C++ header files (.hpp files). If you do not want to modify the “buildrun” script, all includes must be placed in the same directory as the “matrix_multiplication.cpp” file. The build command only looks for user includes in that directory.
- The starter code does not use multithreading and/or vectorization. Parallelism is achieved by running multiple MPI processes on a single and/or multiple hosts.

Running the starter code

1. Assuming you copied the “a1_starter” code in your ECE home directory, i.e., “/home/userID/a1_starter”, cd into the “a1_starter” directory
2. The “input” directory contains several input matrix pairs (identified by the letters a, b in the filename)
3. Run “sh generate_ground_truth.sh”: this will generate the ground truth (result of matrix multiplication) in the “ground_truth” directory (the input is one of the matrix pairs in the “input” directory)
4. Next, run “sh buildrun_standalone.sh”: this script first builds the C++ MPI application and creates the “output” directory (where the output of MPI matrix multiplication are temporarily stored).
5. Then the test cases are run. Correctness is determined by verifying the output matrix of each test case against the ground truth (available in the “ground_truth” directory).
6. The file “output/result.txt” contains experiment details (number of processes, input size etc.) and result (time-to-solution, correctness etc.) of each test case.

Running the starter code

7. The ground truth (output of matrix multiplication) is computed using NumPy.
8. If you want to compare the output of your solution with the ground truth using the diff command, use the “-b” option to ignore white space change: “diff -b file_1, file_2”.
9. The “--host” option of “mpirun” sets the number of “slots” on a compute node which indicates how many processes can potentially execute on that node. For example, “--host \$HOSTANME:16” means up to 16 MPI processes are allowed on the node. The number of slots should not be significantly larger than the number of CPU-cores available on a node.

Interpreting output of the starter code

- “output/result.txt” contains output of all test cases

Example:

exptype, numproc, mtxorder, computetime, totaltime, correct

t, 16, 256, 0.0630541, 0.152243, 1

t, 16, 512, 0.348331, 0.612397, 1

t, 16, 1024, 7.59186, 8.63896, 1

s, 4, 1024, 7.59362, 8.64095, 1

s, 16, 1024, 7.60069, 8.64888, 1

- exptype: t - time-to-solution experiment, s - strong scaling experiment
- numproc: number of MPI processes used
- mtxorder: order of input matrix (input size)
- computetime: totaltime - the fraction of time spent in writing output to file
- output correct: 1, output incorrect: 0 (determined by comparing with the ground truth)

Standalone vs. multi-node experiments

- The “`buildrun_standalone.sh`” script is for running experiment on a single host (with multiple CPUs/cores, e.g., ecetesla hosts have up to 24 CPUs).
- The same MPI application can be run on both a single host and multiple interconnected hosts, no code recompilation is required.
- Many parallel applications benefit from access to a large number of distributed processes. At the same time, performance of distributed processing is heavily influenced by the capacity of the interconnect (network) of a distributed platform.
- We set up MPI in the ecehadoop cluster. Each ecehadoop node, however, only has a few cores, so we are not expecting any significant gain compared to running on a single host, e.g., on ecetesla using 16 processes.
- A second “`buildrun`” script for running your code on ecehadoop is available. It has the same test cases (as in “`buildrun_standalone.sh`”), however, runs in a multi-node setting.
- The only modification in the second “`buildrun`” script is a list of hosts, on which an MPI application is run, as an argument to the “`mpirun`” command.

Multi-node experiments

- Log in to “ecehadoop”.
- Run the following script: “sh copy_ssh_public_key.sh”. (Run this script only once.)
- Then use the following script to run an MPI application on the ecehadoop cluster: “sh buildrun_multinode.sh “. The “mpi_ecehadoop_hosts” is the “hostfile” for the “[mpirun](#)” command.
- Note: If you notice the MPI application is not progressing, make sure you can ssh to the hosts listed in the file “ecehadoop_hosts” without entering password.
- Note: The following warning message can be ignored; it does not interfere with application execution or output. “Authorization required, but no authorization protocol specified”. (ECE sysadmins do not know how to fix this yet.)

Apache Spark™

- We will create accounts on eceHadoop for everyone in the class.
- The following set of slides describe how to run a Spark application on ecehadoop cluster.

Using the starter code

- The [starter code](#) includes an implementation of word counting in Spark. Shell scripts are provided to build and run the code.

Running the starter code

Please follow the steps below to run the sample Spark (scala) word counting program (available in the starter code tarball):

`ssh userID@eceTerm3.uwaterloo.ca` (userID is your ECE alphanumeric username)

`ssh userID@eceHadoop.private.uwaterloo.ca` // this is the eceHadoop master node (it runs the Spark driver program)

Make sure the following command completes without any error:

`/opt/hadoop-latest/hadoop/bin/hdfs dfs -ls "/user/userID/"` // your HDFS home directory

Assuming you copied the a2_starter code in your ECE home directory, i.e., `/home/userID/a2_starter`, cd into the a2_starter directory:

`cd /home/userID/a2_starter`

Now build/run the word counting program:

`sh buildrun_wc_spark_cluster.sh`

Running the starter code

The program will copy input data from a2_starter/sample_input to the HDFS directory /user/userID/

Upon successful completion of the Spark program, the output will be written to the HDFS directory /user/userID/a2_starter_code_output_spark/

Run the following command to view the output written to HDFS:

```
/opt/hadoop-latest/hadoop/bin/hdfs dfs -cat  
"/user/userID/a2_starter_code_output_spark/*"
```

Please do not put your application code/scripts in your HDFS directory, i.e., /user/userID/. This directory is only for input and output of a Spark program.

Cluster house rules

- Storage space on ecehadoop cluster is limited, so please keep your HDFS usage (/user/userID/) below 1GB.
- If you want to experiment with large inputs then consider placing them in HDFS under /tmp where they can be accessed by group members and avoid duplication.
- Please limit yourself to running one Spark job at a time, especially during busy periods such as shortly before the deadline.
- Please do not let any job run for more than 30 minutes on the cluster.

External computing resources

- [AMD University Program AI & HPC Cluster](#)
- [Microsoft Azure for Students](#)
- [AWS](#)
- [Google Colab](#)
- [IBM Cloud](#)
- [Oracle Cloud](#)
- [Sharcnet Canada](#)